

Exercise 2: E-commerce Platform Search Function

```
public class SearchFunction {  
  
    // Product Class  
    static class Product {  
        int productId;  
        String productName;  
        String category;  
  
        public Product(int productId, String productName, String category) {  
            this.productId = productId;  
            this.productName = productName;  
            this.category = category;  
        }  
  
        @Override  
        public String toString() {  
            return "ID: " + productId +  
                ", Name: " + productName +  
                ", Category: " + category;  
        }  
    }  
  
    // Linear Search Method  
    public static Product linearSearch(Product[] products, int id) {  
        for (Product product : products) {  
            if (product.productId == id) {  
                return product;  
            }  
        }  
        return null;  
    }  
}
```

```
// Binary Search Method
```

```
public static Product binarySearch(Product[] products, int id) {
```

```
    int left = 0;
```

```
    int right = products.length - 1;
```

```
    while (left <= right) {
```

```
        int mid = (left + right) / 2;
```

```
        if (products[mid].productId == id) {
```

```
            return products[mid];
```

```
        }
```

```
        if (products[mid].productId < id) {
```

```
            left = mid + 1;
```

```
        } else {
```

```
            right = mid - 1;
```

```
        }
```

```
    }
```

```
    return null;
```

```
}
```

```
// Main Method
```

```
public static void main(String[] args) {
```

```
    // Sorted array of products
```

```
    Product[] products = {
```

```
        new Product(101, "Laptop", "Electronics"),
```

```
        new Product(102, "Mobile", "Electronics"),
```

```
        new Product(103, "Shoes", "Fashion"),
```

```
        new Product(104, "Watch", "Accessories"),
```

```
        new Product(105, "Bag", "Fashion")
    };

    int searchId = 104;

    // Linear Search
    Product result1 = linearSearch(products, searchId);

    System.out.println("Linear Search Result:");
    if (result1 != null)
        System.out.println(result1);
    else
        System.out.println("Product not found");

    // Binary Search
    Product result2 = binarySearch(products, searchId);

    System.out.println("\nBinary Search Result:");
    if (result2 != null)
        System.out.println(result2);
    else
        System.out.println("Product not found");

    // Complexity Analysis
    System.out.println("\nTime Complexity Analysis:");
    System.out.println("Linear Search -> Best: O(1), Average: O(n), Worst: O(n)");
    System.out.println("Binary Search -> Best: O(1), Average: O(log n), Worst: O(log n)");
}
}
```

Linear Search Result:

ID: 104, Name: Watch, Category: Accessories

Binary Search Result:

ID: 104, Name: Watch, Category: Accessories

Time Complexity Analysis:

Linear Search -> Best: $O(1)$, Average: $O(n)$, Worst: $O(n)$

Binary Search -> Best: $O(1)$, Average: $O(\log n)$, Worst: $O(\log n)$

Exercise 7: Financial Forecasting

```
public class FinancialForecast {

    // Recursive method to calculate future value
    public static double predictFutureValue(double currentValue, double growthRate, int years) {

        // Base case
        if (years == 0) {
            return currentValue;
        }

        // Recursive case
        return predictFutureValue(currentValue, growthRate, years - 1) * (1 + growthRate);
    }

    public static void main(String[] args) {

        double currentValue = 10000; // Initial investment
        double growthRate = 0.10;    // 10% annual growth
        int years = 5;

        double futureValue = predictFutureValue(currentValue, growthRate, years);

        System.out.println("Current Value: ₹" + currentValue);
        System.out.println("Growth Rate: " + (growthRate * 100) + "%");
        System.out.println("Years: " + years);
        System.out.printf("Predicted Future Value: ₹%.2f%n", futureValue);

        // Analysis
        System.out.println("\nTime Complexity: O(n)");
    }
}
```

```
        System.out.println("Space Complexity: O(n) due to recursive call stack.");  
    }  
}
```

```
Current Value: ₹10000.0  
Growth Rate: 10.0%  
Years: 5  
Predicted Future Value: ₹16105.10  
  
Time Complexity: O(n)  
Space Complexity: O(n) due to recursive call stack.
```